

Module	Description	Example	Script
anthropic	creating a client	client = Anthropic(api_key=apikey)	g20/demo.py
anthropic	importing the module	from anthropic import Anthropic	g20/demo.py
core	api, reading a key from a file	apikey = fh.readline().strip()	g19/demo.py
core	class, accessing the instance	cols = self.columns	g20/demo.py
core	class, defining a class attribute	sig = 'Howdy'	g20/demo.py
core	class, defining a method	def capcols(self):	g20/demo.py
core	class, extending existing class	class myDF(pd.DataFrame):	g20/demo.py
core	continue, going on to next loop item	continue	g06/demo.py
core	dictionary, adding a new entry	co['po'] = 'CO'	g05/demo.py
core	dictionary, creating	co = {'name':'Colorado', 'capital':'Denver'}	g05/demo.py
core	dictionary, creating via comprehension	word_lengths = { w:len(w) for w in wordlist }	g06/demo.py
core	dictionary, iterating through key-value pairs	for w,l in word_lengths.items():	g06/demo.py
core	dictionary, looking up a value	name = ny['name']	g05/demo.py
core	dictionary, making a list of	list1 = [co,ny]	g05/demo.py
core	dictionary, obtaining a list of keys	names = super_dict.keys()	g05/demo.py
core	f-string, grouping with commas	print(f'Total population: {tot_pop:,}')	g12/demo.py
core	f-string, using a formatting string	print(f"PV of {payment} with T={year} and r={r} is \${p..."	g08/demo.py
core	file, closing	fh.close()	g02/demo.py
core	file, opening for reading	fh = open('states.csv')	g05/demo.py
core	file, opening for writing	fh = open(filename,"w")	g02/demo.py
core	file, output using print	print("It was written during",year,file=fh)	g02/demo.py
core	file, output using write	fh.write("Where was this file was written?\n")	g02/demo.py
core	file, print without adding spaces	print('\nOuter:\n', join_o['_merge'].value_counts(), s...	g15/demo.py
core	file, reading one line at a time	for line in fh:	g05/demo.py
core	for, looping through a list	for n in a_list:	g04/demo.py
core	for, looping through a list of tuples	for number,name in div_info:	g14/demo.py
core	function, calling	d1_ssqr = sumsq(d1)	g07/demo.py
core	function, calling with an optional argument	sample_function(100, 10, r=0.07)	g08/demo.py
core	function, defining	def sumsq(values: list) -> float:	g07/demo.py

Module	Description	Example	Script
core	function, defining with optional argument	<code>def sample_function(payment:float,year:int,r:float=0.05...</code>	g08/demo.py
core	function, returning a result	<code>return values</code>	g07/demo.py
core	function, using type hinting	<code>def readlist(filename: str) -> list:</code>	g07/demo.py
core	if, starting a conditional block	<code>if l == 5:</code>	g06/demo.py
core	if, using an elif statement	<code>elif s.isalpha():</code>	g06/demo.py
core	if, using an else statement	<code>else:</code>	g06/demo.py
core	lambda function	<code>counts = dat.apply(lambda x:len(x) if type(x)==list el...</code>	g25/demo.py
core	list, appending an element	<code>a_list.append("four")</code>	g03/demo.py
core	list, create via comprehension	<code>cubes = [n**3 for n in a_list]</code>	g04/demo.py
core	list, creating	<code>a_list = ["zero", "one", "two", "three"]</code>	g03/demo.py
core	list, determining length	<code>n = len(b_list)</code>	g03/demo.py
core	list, extending with another list	<code>a_list.extend(a_more)</code>	g03/demo.py
core	list, generating a sequence	<code>b_list = range(1,6)</code>	g04/demo.py
core	list, joining with spaces	<code>a_string = " ".join(a_list)</code>	g03/demo.py
core	list, selecting an element	<code>print(a_list[0])</code>	g03/demo.py
core	list, selecting elements 0 to 3	<code>print(a_list[:4])</code>	g03/demo.py
core	list, selecting elements 1 to 2	<code>print(a_list[1:3])</code>	g03/demo.py
core	list, selecting elements 1 to the end	<code>print(a_list[1:])</code>	g03/demo.py
core	list, selecting last 3 elements	<code>print(a_list[-3:])</code>	g03/demo.py
core	list, selecting the last element	<code>print(a_list[-1])</code>	g03/demo.py
core	list, sorting	<code>c_sort = sorted(b_list)</code>	g03/demo.py
core	list, sorting with custom key	<code>bylast = sorted(names, key=lambda x: x.split()[-1])</code>	g25/demo.py
core	list, summing	<code>total = sum(numbers)</code>	g06/demo.py
core	math, raising a number to a power	<code>a_cubes.append(n**3)</code>	g04/demo.py
core	math, rounding a number	<code>rounded = round(ratio,2)</code>	g05/demo.py
core	sets, computing difference	<code>print(name_states - pop_states)</code>	g14/demo.py
core	sets, creating	<code>name_states = set(name_data['State'])</code>	g14/demo.py
core	sets, of tuples	<code>tset1 = set([(1,2), (2,3), (1,3), (2,3)])</code>	g14/demo.py
core	string, concatenating	<code>name = s1+" "+s2+" "+s3</code>	g02/demo.py
core	string, convert to lower case	<code>lower = [s.lower() for s in wordlist]</code>	g06/demo.py
core	string, convert to title case	<code>new_s = s.title()</code>	g06/demo.py

Module	Description	Example	Script
core	string, converting to an int	value = int(s)	g06/demo.py
core	string, creating	filename = "demo.txt"	g02/demo.py
core	string, finding starting index	mm_start = long_string.find("mm")	g06/demo.py
core	string, including a newline character	fh.write(name+"!\n")	g02/demo.py
core	string, is entirely numeric	if s.isnumeric():	g06/demo.py
core	string, matching a substring	has_ñ = [s for s in lower if "ñ" in s]	g06/demo.py
core	string, matching end	a_end = [s for s in lower if s.endswith("a")]	g06/demo.py
core	string, matching multiple starts	ab_start = [s for s in lower if s.startswith(starters)]	g06/demo.py
core	string, matching partial string	is_gas = trim['DBA Name'].str.contains('XPRESS')	g31/demo.py
core	string, matching start	a_start = [s for s in lower if s.startswith("a")]	g06/demo.py
core	string, replacing a substring	words = s.replace(" ", " ").split()	g06/demo.py
core	string, splitting on a comma	parts = line.split(',')	g05/demo.py
core	string, splitting on whitespace	b_list = b_string.split()	g03/demo.py
core	string, stripping blank space	clean = [item.strip() for item in parts]	g05/demo.py
core	string, using the format method	my_ask = prompt.format(tidy)	g20/demo.py
core	tuple, creating	starters = ("a", "b", "0")	g06/demo.py
core	type, obtaining for a variable	print('\nraw_states is a DataFrame object:', type(raw_...	g10/demo.py
csv	setting up a DictReader object	reader = csv.DictReader(fh)	g09/demo.py
geopandas	adding a heatmap legend	slices.plot('s_pop', edgecolor='yellow', linewidth=0.2, le...	g30/demo.py
geopandas	building points from lat, lon	geom = gpd.points_from_xy(adds['lon'], adds['lat'])	g32/demo.py
geopandas	clip a layer	zips_clip = zips.clip(county, keep_geom_type=True)	g26/demo.py
geopandas	combine all geographies in a layer	water_dis = water_by_name.dissolve()	g26/demo.py
geopandas	combine geographies by attribute	water_by_name = water.dissolve('FULLNAME')	g26/demo.py
geopandas	computing areas	zips['z_area'] = zips.area	g30/demo.py
geopandas	construct a buffer	near_water = water_dis.buffer(1600)	g26/demo.py
geopandas	constructing centroids	centroids['geometry'] = tracts.centroid	g31/demo.py
geopandas	drawing a heatmap	near_wv.plot("mil", cmap='Blues', legend=True, ax=ax)	g24/demo.py
geopandas	extracting geometry from a geodataframe	wv_geo = wv['geometry']	g24/demo.py
geopandas	importing the module	import geopandas as gpd	g23/demo.py
geopandas	join to nearest object	served_by = centroids.sjoin_nearest(geo, how='left', dist...	g31/demo.py
geopandas	list layers in a geopackage	layers = gpd.list_layers(demo_file)	g26/demo.py
geopandas	merging data onto a geodataframe	conus = conus.merge(trim, on='STATEFP', how='left', valida...	g24/demo.py
geopandas	obtaining coordinates	print('Number of points:', len(wv_geo.exterior.coords)...	g24/demo.py

Module	Description	Example	Script
geopandas	overlaying a layer using union	slices = zips.overlay(county,how='union',keep_geom_type=...	g30/demo.py
geopandas	plot with categorical coloring	sel.plot('NAME',cmap='Dark2',ax=ax)	g24/demo.py
geopandas	plotting a boundary	syr.boundary.plot(color='gray',linewidth=1,ax=ax)	g23/demo.py
geopandas	project a layer	county = county.to_crs(epsg=utm18n)	g26/demo.py
geopandas	reading a file	syr = gpd.read_file("tl_2016_36_place-syracuse.zip")	g23/demo.py
geopandas	reading a shapefile	states = gpd.read_file("cb_2024_us_state_500k.zip")	g24/demo.py
geopandas	reading data in WKT format	coords = gpd.GeoSeries.from_wkt(big['Georeference'])	g31/demo.py
geopandas	setting the color of a plot	county.plot(color='tan',ax=ax)	g26/demo.py
geopandas	setting transparency via alpha	near_clip.plot(alpha=0.25,ax=ax)	g26/demo.py
geopandas	spatial join, contains	c_contains_z = county.sjoin(zips,how='right',predicate=...	g28/demo.py
geopandas	spatial join, crosses	i_crosses_z = inter.sjoin(zips,how='right',predicate='c...	g28/demo.py
geopandas	spatial join, intersects	z_intersect_c = zips.sjoin(county,how='left',predicate=...	g28/demo.py
geopandas	spatial join, overlaps	z_overlaps_c = zips.sjoin(county,how='left',predicate='...	g28/demo.py
geopandas	spatial join, touches	z_touch_c = zips.sjoin(county,how='left',predicate='tou...	g28/demo.py
geopandas	spatial join, within	z_within_c = zips.sjoin(county,how='left',predicate='wi...	g28/demo.py
geopandas	testing if rows touch a geometry	touches_wv = conus.touches(wv_geo)	g24/demo.py
geopandas	writing a layer to a geodatabase	conus.to_file("conus.gpkg",layer="states")	g24/demo.py
json	importing the module	import json	g05/demo.py
json	using to print an object nicely	print(json.dumps(list1,indent=4))	g05/demo.py
matplotlib	axes, adding a horizontal line	ax21.axhline(medians['etr'], c='r', ls='-', lw=1)	g13/demo.py
matplotlib	axes, adding a vertical line	ax21.axvline(medians['inc'], c='r', ls='-', lw=1)	g13/demo.py
matplotlib	axes, labeling the X axis	ax.set_xlabel('Millions')	g12/demo.py
matplotlib	axes, labeling the Y axis	ax.set_ylabel('Millions')	g12/demo.py
matplotlib	axes, turning off a label	ax.set_ylabel(None)	g14/demo.py
matplotlib	axis, turning off	ax.axis('off')	g30/demo.py
matplotlib	changing marker shape	geo.plot(color='red', marker='D', markersize=20, ax=ax)	g32/demo.py
matplotlib	changing marker size	geo.plot(color='blue',markersize=1,ax=ax1)	g31/demo.py
matplotlib	colors, xkcd palette	syr.plot(color='xkcd:lightblue',ax=ax)	g23/demo.py
matplotlib	figure, adding a title	fig2.suptitle('Pooled Data')	g13/demo.py
matplotlib	figure, four panel grid	fig3, axs = plt.subplots(2,2,sharex=True,sharey=True)	g13/demo.py
matplotlib	figure, left and right panels	fig2, (ax21,ax22) = plt.subplots(1,2)	g13/demo.py
matplotlib	figure, saving	fig.savefig('figure.png')	g12/demo.py
matplotlib	figure, setting the size	fig, axs = plt.subplots(1,2,figsize=(12,6))	g21/demo.py
matplotlib	figure, tuning the layout	fig.tight_layout()	g12/demo.py
matplotlib	figure, working with a list of axes	for ax in axs:	g21/demo.py

Module	Description	Example	Script
matplotlib	importing pyplot	<code>import matplotlib.pyplot as plt</code>	g12/demo.py
matplotlib	setting a linewidth	<code>us.boundary.plot(color='black', linewidth=0.4, ax=ax)</code>	g32/demo.py
matplotlib	setting an edge color	<code>slices.plot('COUNTYFP', edgecolor='yellow', linewidth=0.2, ...</code>	g30/demo.py
matplotlib	setting the default resolution	<code>plt.rcParams['figure.dpi'] = 300</code>	g12/demo.py
matplotlib	using subplots to set up a figure	<code>fig, ax = plt.subplots()</code>	g12/demo.py
os	delete a file	<code>os.remove(out_file)</code>	g26/demo.py
os	importing the module	<code>import os</code>	g19/demo.py
os	test if a file or directory exists	<code>if os.path.exists('apikey.txt'):</code>	g19/demo.py
pandas	RE, replacing a digit or space	<code>unit_part = values.str.replace(r'\d\s', "", regex=True)</code>	g25/demo.py
pandas	RE, replacing a non-digit or space	<code>value_part = values.str.replace(r'\D\s', "", regex=True)</code>	g25/demo.py
pandas	RE, replacing a non-word character	<code>units = units.str.replace(r'\W', "", regex=True)</code>	g25/demo.py
pandas	categorical variable, categories	<code>print(cat.categories)</code>	g13/demo.py
pandas	categorical variable, codes	<code>tidy_data['cat_codes'] = cat.codes</code>	g13/demo.py
pandas	categorical variable, creating	<code>cat = pd.Categorical(tidy_data['type'])</code>	g13/demo.py
pandas	columns, dividing along index	<code>by_day_pct = 100*by_day_use.div(by_day_tot,axis='index'...</code>	g18/demo.py
pandas	columns, dividing with explicit alignment	<code>normed2 = 100*states.div(pa_row,axis='columns')</code>	g10/demo.py
pandas	columns, listing names	<code>print('\nColumns:', list(raw_states.columns))</code>	g10/demo.py
pandas	columns, renaming	<code>county = county.rename(columns={'B01001_001E':'pop'})</code>	g11/demo.py
pandas	columns, retrieving one by name	<code>pop = states['pop']</code>	g10/demo.py
pandas	columns, retrieving several by name	<code>print(pop[some_states]/1e6)</code>	g10/demo.py
pandas	dataframe, appending	<code>gen_all = pd.concat([gen_oswego, gen_onondaga])</code>	g16/demo.py
pandas	dataframe, boolean row selection	<code>print(trim[has_AM], "\n")</code>	g13/demo.py
pandas	dataframe, dropping a column	<code>both = both.drop(columns='_merge')</code>	g16/demo.py
pandas	dataframe, dropping duplicates	<code>flood = flood.drop_duplicates(subset='TAX_ID')</code>	g15/demo.py
pandas	dataframe, dropping missing data	<code>merged = geocodes.dropna()</code>	g12/demo.py
pandas	dataframe, finding duplicate records	<code>dups = parcels.duplicated(subset='TAX_ID', keep=False...</code>	g15/demo.py
pandas	dataframe, getting a block of rows via index	<code>sel = merged.loc[number]</code>	g14/demo.py
pandas	dataframe, inner 1:1 merge	<code>join_i = parcels.merge(flood, how='inner', on="TAX_ID",...</code>	g15/demo.py
pandas	dataframe, inner join	<code>merged = name_data.merge(pop_data,left_on="State",right...</code>	g14/demo.py
pandas	dataframe, iterating through rows	<code>for idx,rec in sample.iterrows():</code>	g20/demo.py
pandas	dataframe, left 1:1 merge	<code>join_l = parcels.merge(flood, how='left', on="TAX_ID",...</code>	g15/demo.py
pandas	dataframe, left m:1 merge	<code>both = gen_all.merge(plants, how='left', on='Plant Code...</code>	g16/demo.py

Module	Description	Example	Script
pandas	dataframe, making a copy	<code>trim = trim.copy()</code>	<code>g13/demo.py</code>
pandas	dataframe, melting	<code>long_form = means.reset_index().melt(id_vars='month')</code>	<code>g18/demo.py</code>
pandas	dataframe, outer 1:1 merge	<code>join_o = parcels.merge(flood, how='outer', on="TAX_ID",...</code>	<code>g15/demo.py</code>
pandas	dataframe, pivoting	<code>by_day_use = usage.pivot(index=['month','day'],columns=...</code>	<code>g18/demo.py</code>
pandas	dataframe, reading tab-delimited data	<code>raw = pd.read_csv(input_file,</code>	<code>g20/demo.py</code>
pandas	dataframe, reading zipped pickle format	<code>sample2 = pd.read_pickle('sample.pkl.zip')</code>	<code>g17/demo.py</code>
pandas	dataframe, resetting the index	<code>hourly = hourly.reset_index()</code>	<code>g18/demo.py</code>
pandas	dataframe, right 1:1 merge	<code>join_r = parcels.merge(flood, how='right', on="TAX_ID",...</code>	<code>g15/demo.py</code>
pandas	dataframe, saving in zipped pickle format	<code>sample.to_pickle('sample.pkl.zip')</code>	<code>g17/demo.py</code>
pandas	dataframe, selecting rows by list indexing	<code>print(low_to_high[-5:])</code>	<code>g10/demo.py</code>
pandas	dataframe, selecting rows via boolean	<code>dup_rec = flood[dups]</code>	<code>g15/demo.py</code>
pandas	dataframe, selecting rows via query	<code>trimmed = county.query("state == '04' or state == '36' ")</code>	<code>g11/demo.py</code>
pandas	dataframe, selective drop of missing data	<code>trim = demo.dropna(subset="Days")</code>	<code>g13/demo.py</code>
pandas	dataframe, set index keeping the column	<code>states = states.set_index('STUSPS',drop=False)</code>	<code>g24/demo.py</code>
pandas	dataframe, shape attribute	<code>print('number of rows, columns:', conus.shape)</code>	<code>g24/demo.py</code>
pandas	dataframe, sorting by a column	<code>county = county.sort_values('pop')</code>	<code>g11/demo.py</code>
pandas	dataframe, sorting by index	<code>summary = summary.sort_index(ascending=False)</code>	<code>g16/demo.py</code>
pandas	dataframe, summing a boolean	<code>print('\nduplicate parcels:', dups.sum())</code>	<code>g15/demo.py</code>
pandas	dataframe, summing across columns	<code>by_day_tot = by_day_use.sum(axis='columns')</code>	<code>g18/demo.py</code>
pandas	dataframe, unstacking an index level	<code>bymo = bymo.unstack('month')</code>	<code>g18/demo.py</code>
pandas	dataframe, using a multilevel column index	<code>means = grid['mean']</code>	<code>g21/demo.py</code>
pandas	dataframe, using xs to select a subset	<code>print(county.xs('04',level='state'))</code>	<code>g11/demo.py</code>
pandas	dataframe, using xs with columns	<code>c1 = grid.xs('c1',axis='columns',level=1)</code>	<code>g21/demo.py</code>
pandas	dataframe, writing as an Excel file	<code>trim.to_excel(output_file,index=False)</code>	<code>g20/demo.py</code>
pandas	dataframe, writing to a CSV file	<code>merged.to_csv('demo-merged.csv')</code>	<code>g14/demo.py</code>
pandas	datetime, building via to_datetime()	<code>date = pd.to_datetime(recs['ts'])</code>	<code>g15/demo.py</code>
pandas	datetime, building with a format	<code>ymd = pd.to_datetime(sample['TRANSACTION_DT'], format=...</code>	<code>g17/demo.py</code>
pandas	datetime, extracting day attribute	<code>recs['day'] = date.dt.day</code>	<code>g15/demo.py</code>
pandas	datetime, extracting hour attribute	<code>recs['hour'] = date.dt.hour</code>	<code>g15/demo.py</code>
pandas	general, display information about object	<code>sample.info()</code>	<code>g17/demo.py</code>
pandas	general, displaying all columns	<code>pd.set_option('display.max_columns',None)</code>	<code>g17/demo.py</code>
pandas	general, displaying all rows	<code>pd.set_option('display.max_rows', None)</code>	<code>g10/demo.py</code>
pandas	general, importing the module	<code>import pandas as pd</code>	<code>g10/demo.py</code>
pandas	general, using copy_on_write mode	<code>pd.options.mode.copy_on_write = True</code>	<code>g17/demo.py</code>
pandas	general, using qcut to create deciles	<code>dec = pd.qcut(county['pop'], 10, labels=range(1,11))</code>	<code>g11/demo.py</code>

Module	Description	Example	Script
pandas	groupby, cumulative sum within group	<code>cumulative_inc = group_by_state['pop'].cumsum()</code>	<code>g11/demo.py</code>
pandas	groupby, descriptive statistics	<code>inc_stats = group_by_state['pop'].describe()</code>	<code>g11/demo.py</code>
pandas	groupby, iterating over groups	<code>for t,g in group_by_state:</code>	<code>g11/demo.py</code>
pandas	groupby, median of each group	<code>pop_med = group_by_state['pop'].median()</code>	<code>g11/demo.py</code>
pandas	groupby, quantile of each group	<code>pop_25th = group_by_state['pop'].quantile(0.25)</code>	<code>g11/demo.py</code>
pandas	groupby, return group number	<code>groups = group_by_state.ngroup()</code>	<code>g11/demo.py</code>
pandas	groupby, return number within group	<code>seqnum = group_by_state.cumcount()</code>	<code>g11/demo.py</code>
pandas	groupby, return rank within group	<code>rank_pop = group_by_state['pop'].rank()</code>	<code>g11/demo.py</code>
pandas	groupby, select first records	<code>first2 = group_by_state.head(2)</code>	<code>g11/demo.py</code>
pandas	groupby, select largest values	<code>largest = group_by_state['pop'].nlargest(2)</code>	<code>g11/demo.py</code>
pandas	groupby, select last records	<code>last2 = group_by_state.tail(2)</code>	<code>g11/demo.py</code>
pandas	groupby, size of each group	<code>num_rows = group_by_state.size()</code>	<code>g11/demo.py</code>
pandas	groupby, sum of each group	<code>state = county.groupby('state')['pop'].sum()</code>	<code>g11/demo.py</code>
pandas	index, creating with 3 levels	<code>county = county.set_index(['state','county', 'NAME'])</code>	<code>g11/demo.py</code>
pandas	index, listing names	<code>print('\nIndex (rows):', list(raw_states.index))</code>	<code>g10/demo.py</code>
pandas	index, renaming values	<code>div_pop = div_pop.rename(index=div_names)</code>	<code>g12/demo.py</code>
pandas	index, retrieving a row by name	<code>pa_row = states.loc['Pennsylvania']</code>	<code>g10/demo.py</code>
pandas	index, retrieving first rows by location	<code>print(low_to_high.iloc[0:10])</code>	<code>g10/demo.py</code>
pandas	index, retrieving last rows by location	<code>print(low_to_high.iloc[-5:])</code>	<code>g10/demo.py</code>
pandas	index, setting to a column	<code>states = raw_states.set_index('name')</code>	<code>g10/demo.py</code>
pandas	plotting, bar plot	<code>reg_pop.plot.bar(title='Population',ax=ax)</code>	<code>g12/demo.py</code>
pandas	plotting, histogram	<code>hh_data['etr'].plot.hist(ax=ax1,bins=20,title='Distribu...</code>	<code>g13/demo.py</code>
pandas	plotting, horizontal bar plot	<code>div_pop.plot.barh(title='Population',ax=ax)</code>	<code>g12/demo.py</code>
pandas	plotting, scatter colored by 3rd var	<code>tidy_data.plot.scatter(ax=ax4,x='Income',y='ETR',c='Reg...</code>	<code>g13/demo.py</code>
pandas	plotting, scatter plot	<code>hh_data.plot.scatter(ax=ax21,x='inc',y='etr',title='ETR...</code>	<code>g13/demo.py</code>
pandas	plotting, turning off legend	<code>sel.plot.barh(x='Name',y='percent',ax=ax,legend=None)</code>	<code>g14/demo.py</code>
pandas	reading, csv data	<code>raw_states = pd.read_csv('state-data.csv')</code>	<code>g10/demo.py</code>
pandas	reading, from an open file handle	<code>gen_oswego = pd.read_csv(fh1)</code>	<code>g16/demo.py</code>
pandas	reading, setting index column	<code>state_data = pd.read_csv('state-data.csv',index_col='na...</code>	<code>g12/demo.py</code>
pandas	reading, using dtype dictionary	<code>county = pd.read_csv('county_pop.csv',dtype=fips)</code>	<code>g11/demo.py</code>
pandas	series, RE at start	<code>is_LD = trim['Number'].str.contains(r"1 2")</code>	<code>g13/demo.py</code>
pandas	series, applying a function to each element	<code>name_clean = name_parts.apply(' '.join)</code>	<code>g25/demo.py</code>

Module	Description	Example	Script
pandas	series, automatic alignment by index	<code>merged['percent'] = 100*merged['pop']/div_pop</code>	<code>g14/demo.py</code>
pandas	series, combining via <code>mask()</code>	<code>mod['comb_mask'] = unit_part.mask(unit_part=="", mod[...</code>	<code>g25/demo.py</code>
pandas	series, combining via <code>where()</code>	<code>mod['comb_where'] = unit_part.where(unit_part!="", mo...</code>	<code>g25/demo.py</code>
pandas	series, contains RE or RE	<code>is_TT = trim['Days'].str.contains(r"Tu Th")</code>	<code>g13/demo.py</code>
pandas	series, contains a plain string	<code>has_AM = trim['Time'].str.contains("AM")</code>	<code>g13/demo.py</code>
pandas	series, contains an RE	<code>has_AMPM = trim['Time'].str.contains("AM.*PM")</code>	<code>g13/demo.py</code>
pandas	series, converting strings to title case	<code>fixname = subset_view['NAME'].str.title()</code>	<code>g17/demo.py</code>
pandas	series, converting to a list	<code>print(name_data['State'].tolist())</code>	<code>g14/demo.py</code>
pandas	series, converting to lower case	<code>name = mod['name'].str.lower()</code>	<code>g25/demo.py</code>
pandas	series, dropping rows using a list	<code>conus = states.drop(not_conus)</code>	<code>g24/demo.py</code>
pandas	series, element-by-element or	<code>is_either = is_ca is_tx</code>	<code>g17/demo.py</code>
pandas	series, filling missing values	<code>mod['comb_units'] = mod['comb_where'].fillna('feet')</code>	<code>g25/demo.py</code>
pandas	series, random sample	<code>vins = vins.sample(frac=0.001,random_state=789)</code>	<code>g20/demo.py</code>
pandas	series, removing spaces	<code>units = units.str.strip()</code>	<code>g25/demo.py</code>
pandas	series, replacing values using a dictionary	<code>units = units.replace(spellout)</code>	<code>g25/demo.py</code>
pandas	series, retrieving an element	<code>print("\nFlorida's population:", pop['Florida']/1e6)</code>	<code>g10/demo.py</code>
pandas	series, sort in decending order	<code>div_pop = div_pop.sort_values(ascending=False)</code>	<code>g12/demo.py</code>
pandas	series, sorting by value	<code>low_to_high = normed['med_pers_inc'].sort_values()</code>	<code>g10/demo.py</code>
pandas	series, splitting strings on whitespace	<code>name_parts = name.str.split()</code>	<code>g25/demo.py</code>
pandas	series, splitting via RE	<code>trim['Split'] = trim["Time"].str.split(r": - ")</code>	<code>g13/demo.py</code>
pandas	series, splitting with <code>expand</code>	<code>exp = trim["Time"].str.split(r": - ", expand=True)</code>	<code>g13/demo.py</code>
pandas	series, summing	<code>reg_pop = by_reg['pop'].sum()/1e6</code>	<code>g12/demo.py</code>
pandas	series, unstacking	<code>tot_wide = tot_amt.unstack('PGI')</code>	<code>g17/demo.py</code>
pandas	series, using <code>isin()</code>	<code>fixed = flood['TAX_ID'].isin(dup_rec['TAX_ID'])</code>	<code>g15/demo.py</code>
pandas	series, using <code>value_counts()</code>	<code>print('\nOuter:\n', join_o['_merge'].value_counts(), s...</code>	<code>g15/demo.py</code>
pydantic	defining an object	<code>class ComplaintInfo(BaseModel):</code>	<code>g20/demo.py</code>
pydantic	importing the module	<code>from pydantic import BaseModel</code>	<code>g20/demo.py</code>
requests	adding headers	<code>response = requests.get(api,payload,headers=headers)</code>	<code>g32/demo.py</code>
requests	calling the <code>get()</code> method	<code>response = requests.get(api,payload)</code>	<code>g19/demo.py</code>
requests	checking the URL	<code>print('url:', response.url)</code>	<code>g19/demo.py</code>
requests	checking the response text	<code>print(response.text)</code>	<code>g19/demo.py</code>
requests	checking the status code	<code>print('status:', response.status_code)</code>	<code>g19/demo.py</code>
requests	decoding a JSON response	<code>rows = response.json()</code>	<code>g19/demo.py</code>
requests	geocoding via nominatim	<code>api = "https://nominatim.openstreetmap.org/search"</code>	<code>g32/demo.py</code>
requests	importing the module	<code>import requests</code>	<code>g19/demo.py</code>

Module	Description	Example	Script
scipy	calling newton's method	<code>cr = opt.newton(find_cube_root,xinit,maxiter=20,args=[y. .</code>	<code>g08/demo.py</code>
scipy	importing the module	<code>import scipy.optimize as opt</code>	<code>g08/demo.py</code>
seaborn	adding a title to a grid object	<code>jg.fig.suptitle('Distribution of Hourly Load')</code>	<code>g18/demo.py</code>
seaborn	barplot	<code>hue='month',palette='deep',ax=ax1)</code>	<code>g18/demo.py</code>
seaborn	basic violin plot	<code>sns.violinplot(data=janjul,x="month",y="usage")</code>	<code>g18/demo.py</code>
seaborn	boxenplot	<code>sns.boxenplot(data=janjul,x="month",y="usage")</code>	<code>g18/demo.py</code>
seaborn	calling tight_layout on a grid object	<code>jg.fig.tight_layout()</code>	<code>g18/demo.py</code>
seaborn	drawing a heatmapped grid	<code>sns.heatmap(means,annot=True,fmt="0f",cmap='Spectral',. . .</code>	<code>g21/demo.py</code>
seaborn	importing the module	<code>import seaborn as sns</code>	<code>g18/demo.py</code>
seaborn	joint distribution hex plot	<code>jg = sns.jointplot(data=bymo,x=1,y=7,kind='hex')</code>	<code>g18/demo.py</code>
seaborn	line plot	<code>sns.lineplot(data=long_form,x='hour',y='value',hue='mon. . .</code>	<code>g18/demo.py</code>
seaborn	setting axis titles on a grid object	<code>jg.set_axis_labels('January','July')</code>	<code>g18/demo.py</code>
seaborn	setting the theme	<code>sns.set_theme(style="white")</code>	<code>g18/demo.py</code>
seaborn	split violin plot	<code>hue="month",palette='deep',split=True)</code>	<code>g18/demo.py</code>
sql	IN clause using a subquery	<code>WHERE PID IN (SELECT . . .)</code>	<code>g27/demo.py</code>
sql	appending to a table via pandas	<code>n = df.to_sql('enrollment',con,if_exists='append',index. . .</code>	<code>g29/demo.py</code>
sql	changing data type with CAST	<code>CAST('Grid kV' AS NUMERIC) AS kV</code>	<code>g27/demo.py</code>
sql	connecting to a SQLite database	<code>con = sqlite3.connect(eia_file)</code>	<code>g27/demo.py</code>
sql	counting records	<code>COUNT(*) AS count</code>	<code>g27/demo.py</code>
sql	create table with primary key	<code>cur = con.execute(create_table)</code>	<code>g29/demo.py</code>
sql	creating a table with a unique constraint	<code>cur = con.executescript(create_enrollment)</code>	<code>g29/demo.py</code>
sql	creating a view	<code>CREATE VIEW summary AS SELECT . . .</code>	<code>g29/demo.py</code>
sql	dropping a table	<code>DROP TABLE IF EXISTS courses</code>	<code>g29/demo.py</code>
sql	executing a SQL script	<code>cur = con.executescript(sql_cmds)</code>	<code>g29/demo.py</code>
sql	insert using a template	<code>INSERT INTO courses VALUES (?,?,?)</code>	<code>g29/demo.py</code>
sql	inserting a single record	<code>cur = con.execute(insert_one)</code>	<code>g29/demo.py</code>
sql	inserting multiple rows via executemany	<code>cur = con.executemany("INSERT INTO courses VALUES (?,?,. . .</code>	<code>g29/demo.py</code>
sql	obtaining a list of tables	<code>SELECT name,sql FROM sqlite_master</code>	<code>g27/demo.py</code>
sql	reading a table via pandas	<code>utility = pd.read_sql("SELECT * FROM Utility;", con)</code>	<code>g27/demo.py</code>
sql	retrieving column names from cursor	<code>cur_info = cur.description</code>	<code>g27/demo.py</code>
sql	retrieving count of rows affected	<code>print('\nRows affected',cur.rowcount)</code>	<code>g29/demo.py</code>
sql	retrieving rows via fetchall	<code>rows = cur.fetchall()</code>	<code>g27/demo.py</code>
sql	select column name with spaces	<code>SELECT . . . 'Energy Source'</code>	<code>g27/demo.py</code>
sql	setting join keys with USING	<code>JOIN . . . USING(UID)</code>	<code>g27/demo.py</code>

Module	Description	Example	Script
sql	simple select of all columns	cur = con.execute("SELECT * FROM Utility;")	g27/demo.py
sql	sort in descending order	ORDER BY total_mw DESC	g27/demo.py
sql	starting a with block	with con:	g29/demo.py
sql	summing a variable	SUM('Nameplate MW') AS mw	g27/demo.py
sql	updating fields for selected records	cur = con.execute(update_cmd)	g29/demo.py
sql	using LIKE in a WHERE clause	WHERE Name LIKE '%national grid%'	g27/demo.py
sql	using fetchone to retrieve one row	check = cur.fetchone()	g29/demo.py
sql	using set in an update	SET name = REPLACE(name, 'Intro', 'Introduction')	g29/demo.py
sql	where with an IN clause	WHERE p.State IN ('VT', 'NH')	g27/demo.py
sys	exiting a script	sys.exit()	g20/demo.py
sys	importing the module	import sys	g20/demo.py
zipfile	importing the module	import zipfile	g16/demo.py
zipfile	opening a file in an archive	fh1 = archive.open('generators-oswego.csv')	g16/demo.py
zipfile	opening an archive	archive = zipfile.ZipFile('generators.zip')	g16/demo.py
zipfile	reading the list of files	print(archive.namelist())	g16/demo.py